

Sistema de notificaciones

Guía técnica: cómo funciona, de principio a fin

jarvis_api · Amazonas365 · Generado: 13/8/2026, 11:30:03 a. m.

Este documento explica el sistema de notificaciones de Jarvis365: dónde nace un aviso, cómo se decide qué dice y quién puede verlo, cómo viaja en tiempo real y cómo se consulta después. Está escrito para poder abrir el código y saber a qué archivo ir.

La tabla de tipos de la sección 5 no está escrita a mano: se genera leyendo el registro real de estrategias del código. Si aparece un tipo nuevo, sale en esta guía la próxima vez que se genere. Ningún aviso puede estar descrito de una forma en el papel y de otra en el sistema.

1. Qué es una notificación

Un HECHO que ya ocurrió y que alguien debe conocer: se creó un establecimiento, cambió tu horario, marcaste tu entrada, alguien comentó tu día.

- Nace SIEMPRE después de que la operación se guardó. Primero se escribe el establecimiento; luego se avisa.
- Nunca puede tumbar lo que la originó. Si falla el aviso, el establecimiento ya está creado y la petición responde igual.
- Se guarda con su texto ya escrito, en español e inglés. No se guarda una plantilla: si mañana se cambia una frase, el pasado tiene que seguir diciendo lo que dijo.

El actor y el recurso se REFERENCIAN y además se COPIAN. La referencia sirve para navegar; la copia, para que dentro de seis meses el aviso siga leyéndose "Daniel Flores desactivó a Francisca Doral" aunque ese establecimiento ya no exista. Sin la copia, el histórico se vacía solo.

2. El recorrido completo

Estos cinco pasos ocurren en orden cada vez que el sistema avisa de algo:



3. Quién puede verla

Es la decisión más delicada del sistema, y se responde DOS veces por dos caminos distintos que tienen que decir lo mismo: al emitir por socket y al consultar la bandeja.

ALCANCE	QUIÉN LO RECIBE	SALA DE SOCKET	SE USA PARA
<code>global</code>	Todos los usuarios	<code>todas</code>	Anuncios de la plataforma y comentarios del horario
<code>personal</code>	Solo los de recipients	<code>user:<id></code>	Tu marcaje, cambios de tu jornada, respuesta a lo que pediste
<code>admin</code>	Solo administradores	<code>admins</code>	Solicitudes de cambio pendientes de aprobar

Una notificación personal NO se emite a todos para esconderla en el cliente. Filtrar en el front no la hace privada: solo la hace invisible, porque el contenido ya viajó. Por eso lo personal va a la sala del destinatario y la consulta filtra en el SERVIDOR, a partir de la sesión. El cliente nunca dice de quién es la consulta.

4. El patrón Estrategia

Cada tipo de notificación se registra a sí mismo y declara cómo se comporta. El servicio no sabe qué dice ninguna, y el cliente no sabe cómo se pinta ninguna: los dos preguntan.

```
registerStrategy('establishment.created', {
  family: 'resource',      // cómo lo pinta el cliente
  scope: 'global',        // quién lo ve
  level: 'success',       // color del punto en la campana
  action: 'created',
  text: (ctx, lang) => ({ title: '...', body: '...' }),
  audience: (ctx) => [...], // solo si scope es 'personal'
});
```

Sin esto, el servicio sería un switch gigante que crece con cada evento del sistema. Con el registro, agregar un aviso nuevo es añadir un objeto: el servicio no se toca.

La familia la manda el backend, no el nombre del tipo

El cliente NO deduce la apariencia partiendo el tipo por el punto. Si lo hiciera, un tipo nuevo obligaría a tocar también el front, y un tipo mal escrito caería en un estilo cualquiera sin avisar. La familia viaja declarada dentro del aviso y el cliente solo obedece.

5. Los tipos que existen hoy

Leídos del registro en el momento de generar este documento: 19 tipos. La columna "Audiencia propia" indica si el tipo calcula sus destinatarios uno por uno.

TIPO	FAMILIA	QUIÉN LO VE	AUDIENCIA PROPIA	NIVEL
attendance.checkIn	attendance	Solo los destinatarios	Sí	info
attendance.checkOut	attendance	Solo los destinatarios	Sí	success
attendance.commented	comment	Todos los usuarios	—	info
establishment.activated	resource	Todos los usuarios	—	success
establishment.created	resource	Todos los usuarios	—	success
establishment.deactivated	resource	Todos los usuarios	—	warning
establishment.deleted	resource	Todos los usuarios	—	danger
establishment.updated	resource	Todos los usuarios	—	info
franchise.created	resource	Todos los usuarios	—	success
franchise.deleted	resource	Todos los usuarios	—	danger
franchise.updated	resource	Todos los usuarios	—	info
overtime.decided	schedule	Solo los destinatarios	Sí	info
schedule.changeApproved	schedule	Solo los destinatarios	Sí	success
schedule.changed	schedule	Solo los destinatarios	Sí	info
schedule.changedAdmin	schedule	Solo los destinatarios	Sí	info
schedule.changeRejected	schedule	Solo los destinatarios	Sí	danger
schedule.changeRequested	schedule	Solo administradores	—	warning
schedule.changeWithdrawn	schedule	Solo los destinatarios	Sí	info

TIPO	FAMILIA	QUIÉN LO VE	AUDIENCIA PROPIA	NIVEL
system.announcement	system	Todos los usuarios	—	success

6. Los endpoints

Todos cuelgan de /api_jarvis/v1 y exigen sesión. Se montan en este orden exacto:

MÉTODO Y RUTA	PARA QUÉ SIRVE	PERMISO
GET /notifications	La bandeja paginada, con el estado de lectura de quien pregunta	Sesión
GET /notifications/unread-count	Solo el número para la campana; no trae documentos	Sesión
POST /notifications/:id/read	Marcar una como leída. Idempotente	Sesión
POST /notifications/read-all	Marcar como leídas todas las que le tocan	Sesión
POST /notifications/:id/decide	Aprobar o rechazar una solicitud de horario	Admin
GET /notifications/schedule/pending	Solicitudes pendientes indexadas por celda, para la grilla	Sesión
POST /notifications/:id/withdraw	Retirar una solicitud propia	Su autor
POST /notifications/announcement	Publicar un anuncio firmado por el usuario del sistema	Admin

El orden de montaje importa: Express resuelve por orden de registro y hay rutas con parámetro (/notifications/:id/read) que podrían capturar a otras. Si se reordenan, una ruta puede responder por otra sin error y sin aviso.

Lo leído vive en otra colección

Una notificación global la reciben todos. Guardar los lectores dentro del documento obligaría a reescribirlo cada vez que alguien abre la campana, y el arreglo crecería sin techo. En su lugar, cada lectura es un documento pequeño e independiente en notificationRead, con índice único por (notificación, usuario): marcar dos veces no duplica ni falla.

7. Mapa de archivos

ARCHIVO	DE QUÉ SE OCUPA
index.js	La puerta del módulo y el mapa del flujo. Empezar a leer por acá
notification.model.js	La forma del aviso guardado
notificationRead.model.js	Quién leyó qué, en colección aparte
notification.service.js	notify(): la mecánica de crear, firmar y emitir
notification.audience.js	Quién lo ve: salas de socket y filtro de consulta
notification.routes.js	Monta los endpoints, en orden
routes/inbox.routes.js	Leer la bandeja y marcar leído

ARCHIVO	DE QUÉ SE OCUPA
<code>routes/requests.routes.js</code>	Solicitudes de horario: decidir, retirar, consultar
<code>routes/announcement.routes.js</code>	Publicar anuncios de la plataforma
<code>strategies/index.js</code>	Carga todas las familias y expone el registro
<code>strategies/registry.js</code>	El mecanismo: registrar y resolver un tipo
<code>strategies/helpers.js</code>	Trozos de frase compartidos por varias familias
<code>strategies/*.strategies.js</code>	Un archivo por familia, con sus tipos y sus textos

8. Cómo agregar un aviso nuevo

- Registra la estrategia en el archivo de su familia, dentro de `strategies/`. Si la familia no existe todavía, crea el archivo e impórtalo en `strategies/index.js` — sin ese `import`, el tipo no se registra y cae en silencio al texto de respaldo.
- Llama a `notify()` donde ocurre el hecho, SIEMPRE después de haberlo guardado.
- Nada más. El guardado, la emisión por socket y la bandeja ya funcionan para el tipo nuevo.

Si la familia visual es nueva, hay que darla de alta también en el `enum` de `notification.model.js` y en el registro de vistas del cliente, que es su espejo.

9. Decisiones y por qué

DECISIÓN	MOTIVO
<code>notify()</code> nunca lanza	El aviso es un efecto secundario de algo que ya se guardó. Si falla, no puede tumbar la operación que lo originó
Texto guardado, no plantilla	Cambiar una frase no debe reescribir lo que el sistema dijo hace seis meses
Actor y recurso copiados	El histórico tiene que sostenerse aunque el establecimiento se borre o la persona cambie de nombre
Lectura en otra colección	Evita reescribir el mismo documento por cada persona que abre la campana
Salas de socket por usuario	Lo privado no viaja. Esconderlo en el cliente no es lo mismo que no enviarlo
Familia declarada por el backend	El cliente no deduce la apariencia del nombre del tipo: un tipo mal escrito caería en cualquier estilo sin avisar
Una familia por archivo	Para responder "qué avisos existen sobre el horario" abriendo un solo archivo